

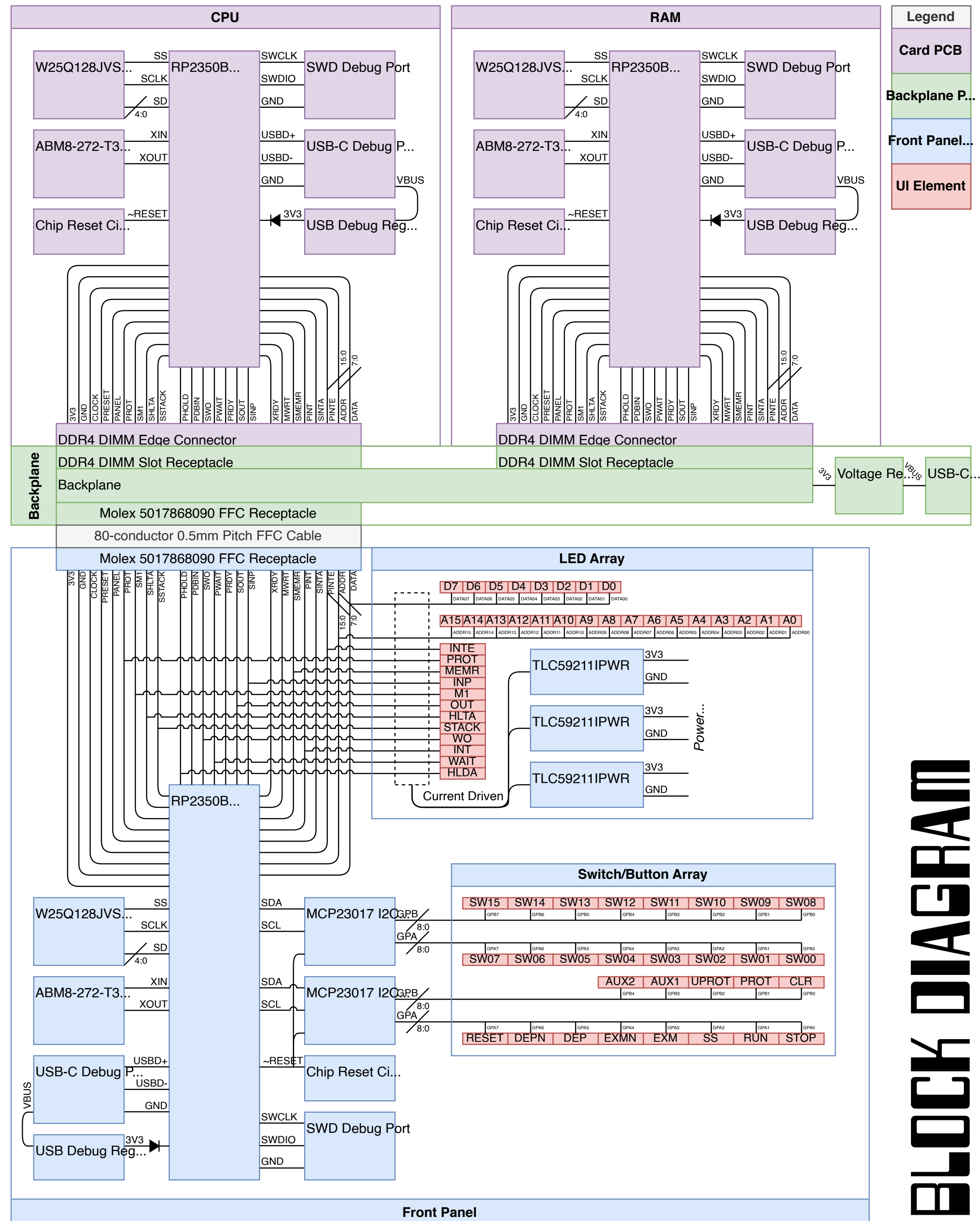
EMULATION

Line	Usage	Pincount
~CLOCK	bus clock	1
A0-A15	Address (CPU driven)	16
A0-D7	Data (tri-state)	8
PINTE	interrupt enable	1
PROT	memory protection	1
SMEMR	memory read strobe	1
SINP	input IO cycle	1
SM1	first machine cycle of instruction flag	1
SOUT	output IO cycle	1
SHLTA	halt	1
SSTACK	stack operation	1
SWO	write-out	1
SINTA	interrupt acknowledge	1
XRDY	memory ready	1
PWAIT	mem/IO wait	1
PRDY	IO ready	1
~PINT	interrupt request	1
~PRESET	reset	1
~PHOLD	"hold" bus control	1
PDBIN	data bus in	1
MWRT	memory write strobe	1
PANEL	front panel data control	1

At the heart of the α Cassiopeiae 8800 lies sophisticated software enabling its functionality. A key component is the custom-built Intel 8080 CPU emulator, developed in Rust. This emulator achieves high fidelity, accurately decoding and executing 254 out of the 256 official 8080 opcodes. It meticulously replicates the 8080's machine cycles, including managing cycle timing, interrupt handling, and respecting I/O wait states, crucial for authentic operation.

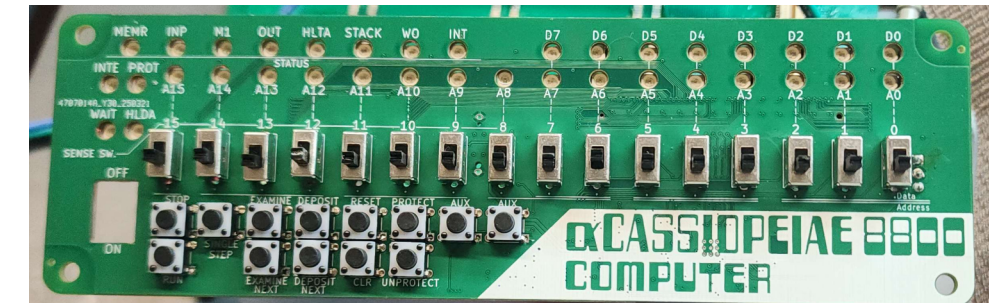
Complementing the emulator are low-level routines leveraging the Raspberry Pi RP2350B's powerful Programmable I/O (PIO) feature. Written in PIO assembly, these routines manage the intricate data exchange across the system's parallel bus. They orchestrate the complex transactions required for communication between the emulated CPU, the RAM card, and the interactive Front Panel, ensuring correct multi-device access and control over the shared bus lines for memory reads/writes, I/O operations, and front panel interactions.

OK CASSIOPEIAE 8800



COMPONENTS

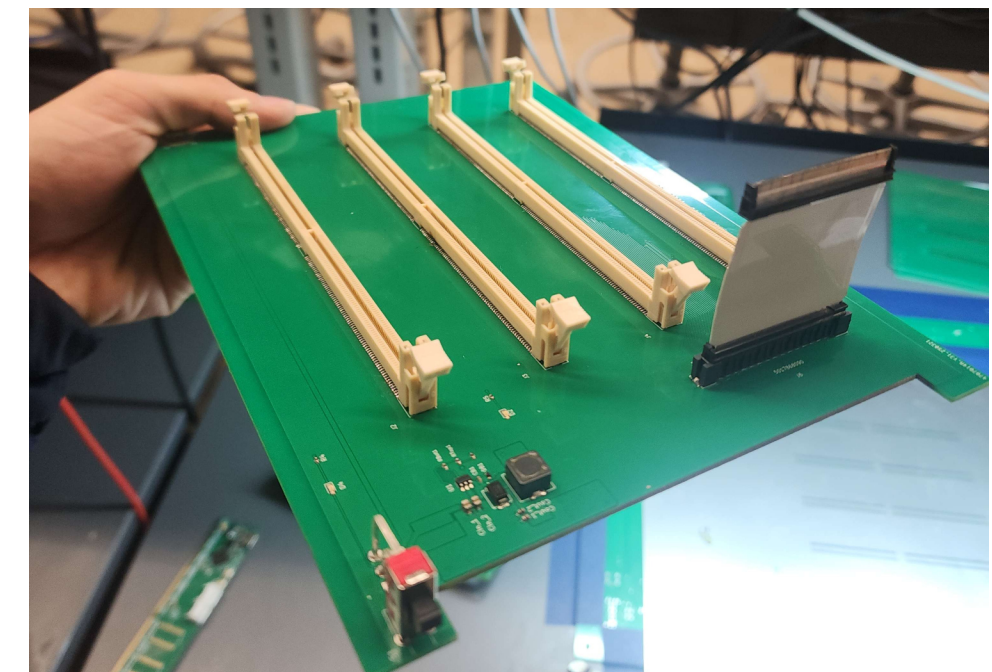
PANEL



CARDS



BACKPLANE



BLOCK DIAGRAM